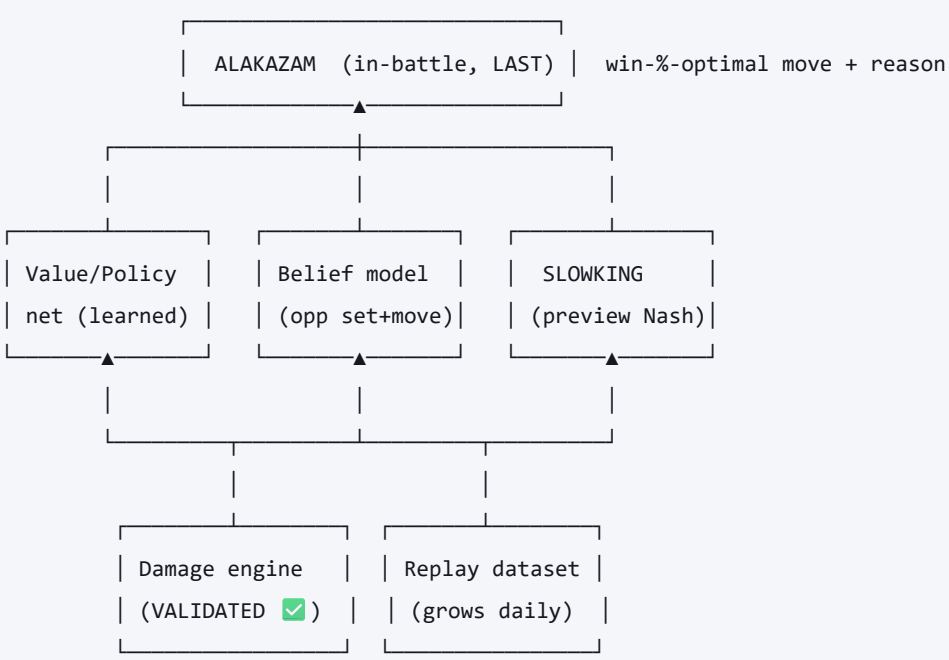
ALAKAZAM v2 — Architecture Spec (the capstone, built last)

Turns `docs/LITERATURE-v2.md` into a concrete, phased build. **ALAKAZAM is the final boss and is built LAST**, because it consumes the other models as inputs. We secure each input — with an honest acceptance bar (proper-score metric + CI + baseline, persisted to JSON, run in CI) — before assembling the capstone.

Guiding principles (from the study):

- **Decisions, not outcomes.** Never predict the match winner; output expected value of decisions.
- **Learn from human data first** (Metamon), **KL-anchor to human play** (CICERO), **mix strategies** (regret matching / R-NaD), **Nash-select for robustness** (AlphaStar/PSRO), **search only to refine** (ReBeL/DeepStack). Present as calibrated EV (sports xG/EPV).
- **Build on the one validated thing** (exact damage). Everything ships with a CI + baseline.

The dependency graph



CHOMP (pivoted) sits alongside SLOWKING (bring/lead EV) and feeds the belief ("what did they bring").

Input models — secure these first (each with an acceptance bar)

Io. Damage engine — ✅ SECURED

- **Is:** validated Gen-9 doubles damage (Champions rules, Serebii-sourced abilities/status).
- **Bar (met):** within 5% of the Smogon damage calculator on 100% of 31 scenarios (`data/damage-validation.json`), gated in CI.
- **Role:** the exact leaf evaluator + rollout simulator everything else uses.

I1. Belief model (opponent set + next-move) — TO BUILD

- **Is:** given the observed battle so far, a **distribution over the opponent's hidden sets** (item/ability/spread/moves) and their **next move**, updated by a Bayesian filter on observed moves + damage, seeded by usage priors (our behaviour-clone).
- **Build:** start from the current clone (top-1 36% / top-3 72%); add (a) **set inference** (narrow the set as moves/items reveal — Champions is open-sheet-ish so this is tractable), (b) a small **move predictor** conditioned on state (active mons, HP, field), trained on replays.
- **Bar:** held-out **top-1/top-3 move-match + cross-entropy vs the clone baseline**, with CI (extend `engine/eval_policy.py`); set-inference measured by log-loss of revealed sets. Must beat the clone.

I2. Value/Policy net (the learned brain) — TO BUILD (biggest piece)

- **Is:** the Metamon lesson — a model trained by **imitation + offline RL** on our human replays (grows with the daily pull) **+ self-play** trajectories, outputting a **value** V(state) and a **policy** π(move|state). This is what makes ALAKAZAM strong without heavy search.
- **Build:** Phase-2a imitation (behaviour cloning of winning play) → Phase-2b offline RL (advantage-weighted / CQL-style) → Phase-2c self-play fine-tune, **KL-anchored to the imitation policy (piKL)** so it stays human-realistic and unexploitable. Leaf value kept **small/fast (NNUE-style)** for later search.
- **Bar:** (a) action-value calibration + top-k on held-out human decisions; (b) **self-play / ladder win-rate vs the behaviour-clone and heuristic baselines**, with CIs — the honest "is it actually better at deciding" test. No match-winner-prediction claim.

I3. SLOWKING (team-preview Nash, outer game) — TO BUILD

- **Is:** at preview, the **bring-4 / lead-2 mixed strategy** and equilibrium value, by solving the matchup as a **matrix game** (regret matching / LP, `engine/slowking/nash.py`) over the **belief** of the opponent's sets — not a greedy best-response (that's what inverted MEDICHAM).
- **Bar:** exploitability of the returned strategy (↓ is better); head-to-head self-play win-rate vs greedy bring. Team-rating (if shown) uses **Nash-averaging**, never scalar Elo.

I4. CHOMP (pivot + prove) — PARALLEL

- **Is:** bring/lead as **expected value over the belief** (xG-for-preview), grounded in validated damage; emits a **calibrated edge** (Kelly/calibration framing), not a win oracle.
- **Bar (the winnable test): do CHOMP's recommended brings beat the human's actual brings**, measured over many held-out games by realized result + a proper score with CI. This is the empirical proof CHOMP is worth it — distinct from (impossible) match prediction.

What we retire/reframe (not delete)

- **JOLTEON** → demoted to a fast **usage-prior / shortlister**; no win-% claim. (Optionally re-tried on a blade-chest/disc class purely as a descriptive meta-rating — low priority.)
- **MEDICHAM win%** → not shown as P(win-the-game); used internally as a **de-biased matchup value** and the rollout simulator. Its damage stays central.
- **DITTO** → **PSRO/double-oracle team-builder** (grow population → best-respond to meta-Nash → Nash-mixture), objective = coverage + validated damage + SLOWKING value, **not** the inverted win%.

Additional inputs the wider study calls for (do it all)

Beyond I0–I4, the literature (poker endgame solving, AlphaStar scouting, CICERO λ-anchoring, sports calibration, VGC-Bench PSRO) implies these. Added to the plan; each ships with metric + CI + baseline.

More input models

- **I5. Meta / matchup model** — (a) usage prior + **P(opponent archetype | what's revealed)** (pivot `engine/archetypes.py`); (b) **learned matchup matrix estimated from REAL game outcomes** (aggregate head-to-head archetype win-rates from the stored games, with Wilson CIs) — this replaces the *biased simulated* payoff matrix in SLOWKING / DITTO / non-transitivity, killing that GIGO at the source. *Bar:* held-out log-loss of the matchup predictions vs a usage baseline.
- **I6. Opponent-type / exploitability model** — infer opponent **skill** (rating + deviation from equilibrium play) and set the **piKL anchor strength λ** accordingly: play near-Nash vs strong opponents (safe/unexploitable), best-respond/exploit vs weak ones (CICERO's DiL-piKL, poker exploitative play, AlphaStar league range). *Bar:* exploit-rate gain vs fixed weak agents in self-play without added exploitability vs strong ones.
- **I7. Endgame exact solver** — when few mons remain (≤2v2, 1v1), the game is small enough to **solve exactly** (retrograde / full matrix-game solve). Gives ground-truth leaf values and clean **training targets for the I2 value net** (the poker endgame-solving trick). *Bar:* exactness vs brute force on toy endgames.
- **I8. Self-play data engine + curation** — the fuel for I2 (Metamon used 5M human + **20M self-play**). Formalize `sim/` into a scalable self-play generator writing to the store schema, plus a **dedup / quality filter** on replays. *Bar:* dataset size + a quality audit (no leakage, balanced, deduped).
- **I-Gimmick. Battle-gimmick module (currently Mega)** — **design principle: model ONE gimmick at a time**, as a **swappable module keyed off** `data/regulations.json`, because each regulation has exactly one active gimmick (Champions Reg M-B = Mega; other regs = Tera or Z-Moves, inactive here). No need to ever carry more than one. For **Mega** specifically: the full forme transform (**stats + types + ability**, not just the ability we did), the **once-per-battle Mega-timing decision**, and a **belief over whether/when the opponent Megas**. **Mega-timing is genuinely strategic, not automatic** — most Pokémon Mega turn 1, but the model must recognize the +EV **hold** cases:
- **speed:** keep a faster **base speed tier** to move first this turn (some Megas are slower, or you want the base speed now and the Mega bulk/power later);
- **base-ability sequencing:** trigger a valuable **base ability first**, then Mega — e.g. Intimidate on entry, **re-set the weather** at the right moment with a base weather-setter, or **farm Moody boosts** (Scovillain runs Moody in base forme, stacking +2s before Mega-ing into Spicy Spray);
- **information: don't reveal the Mega** (its typing/ability) until forced — an info play that ties into the scouting/information-value reward. When the format rotates, drop in a new gimmick module, not a rewrite. *Bar:* correct forme stats/types vs Serebii; does modeling the hold cases beat naive auto-Mega-turn-1 in ALAKAZAM self-play?

Supporting components (required, not standalone models)

- **Calibration layer** — temperature / isotonic on **every** probability we emit (the review's mandate; sports/betting calibration). One utility, applied everywhere, reliability-tested.
- **Information-gain (scouting) reward** — value moves that **reveal** the opponent's hidden set, added to ALAKAZAM's search reward (AlphaStar scouting).
- **Variance / risk estimate** — how much a line depends on rolls / crits / misses → an honest **"how coin-flippy is this"** readout alongside the EV (sports xG smoothing; poker variance).
- **Legality / format checker** — Champions dex + item/species clause, so DITTO/PSRO only generate **legal** teams.
- **Mechanics-coverage tracker** — % of observed moves/abilities the engine models correctly; a live **GIGO gauge** so we always know the simulator's blind spots.

Resolved open question

- **Terastallization: NOT active in Reg M-B** (Serebii/game8 + confirmed) — no Tera model. **Mega Evolution is the only gimmick**; complete the **I-Mega** model instead.

The capstone — ALAKAZAM (built last)

Input: a live position (both actives, HP, field, revealed info) + I1 belief + I2 value/policy + I3 strategic context. **Process:**

1. Form the **public belief state** (I1) over the opponent's hidden info.
2. **Policy proposal** from I2 (fast, KL-anchored to human play).
3. **Depth-limited search** (ReBeL/DeepStack-style) over the **validated damage engine**, solving **each simultaneous turn as a matrix game** (regret matching — kills strategy fusion + speed bias), leaves evaluated by the I2 value net. Reward includes **information-gain** (AlphaStar scouting) so it values forcing reveals.
4. **Output:** the **mixed move recommendation**, the **expected win-% delta** of each option (calibrated EV, not an oracle), and a plain-English **reason** (RAG-of-knowledge, PokéLLMon-style). This is KADABRA fully evolved.

Bar: decision-quality vs held-out human play + **self-play/ladder win-rate vs I2-alone and the behaviour-clone**, with CIs. Search kept only if it beats the no-search policy (Metamon showed it might not — measure honestly). Runs in a **Web Worker/WASM/backend**, never the main thread.

The end-game deliverable — PLAY ALAKAZAM AT A CHOSEN ELO

The capstone is not a number in a table. It is: **a human opens the site and plays a full Champions game against ALAKAZAM, with a difficulty dial set in Elo.** Everything above is the machinery that makes that possible; this is the thing the project is actually for.

Why an Elo dial is the right interface, and why it is more than a slider. A model that only ever plays as hard as it can is useless for training — you lose, learn nothing about *why*, and cannot practise a specific matchup against a specific standard of opponent. The literature is unambiguous that calibrated difficulty is a distinct problem from strength:

- **Maia** (McIlroy-Young et al., KDD 2020) trained *separate* networks per rating band on human chess games and showed each one predicts the moves of players at **that** rating far better than a strong engine does. Weakening a strong engine does **not** reproduce a 1200-rated player — it produces a 2800-rated player making occasional absurd blunders, which is a different and unconvincing thing.
- **AlphaStar** (Vinyals et al., *Nature* 2019) maintained a **league** of distinct agents rather than one optimum, precisely because a single self-play optimum is exploitable and unrepresentative.
- **Metamon** (Grigsby et al., 2025) is the direct precedent in Pokémon: policies trained on human ladder data at different skill strata behave differently, not merely worse.

So the dial must select **between policies trained to imitate a band**, not a single policy with noise injected. ABRA is already positioned for this: the ladder store carries a rating on most games, and MEW's policy layer is swappable (`--policy`), which is exactly the seam a band-conditioned policy plugs into.

Design.

1. **Band the ladder store by rating.** Measure, do not assume, how much data each band has — the statistical-power block already in `eval_harness.py` decides which bands are supportable. Bands with too little data are not offered rather than being offered dishonestly.

2. **Behaviour-clone one policy per band** (the BCSP recipe VGC-Bench found strongest: clone, then self-play from the clone). Bands share the state encoder and the value net; only the policy head differs.
3. **Calibrate the dial empirically.** The label "1500" must be *earned*: the agent set to 1500 plays a large sample against the other bands and against the behaviour clones, and its realised win rates must place it near 1500 on the standard Elo scale. A dial labelled with a rating it has not demonstrated is an asserted constant, which S-standards forbid.
4. **Ship it in the browser.** Inference for a policy of this size is small; search is the expensive part, so the Elo dial can also govern *search depth*, which is a legitimate strength lever on top of the band-conditioned policy — but never a substitute for it.

Acceptance bar. For each offered band: (a) realised Elo within a stated CI of the label, measured against the other bands; (b) move-agreement with held-out human games **at that band** exceeding agreement at every other band — the Maia test, and the one that distinguishes genuine imitation from a handicapped expert; (c) a full game playable in a browser without blocking the main thread.

Honest status. Not started. It depends on the value net (in progress), the belief model, and a self-play corpus large enough to train from. The rating data also has a known problem: this project has already measured that the better-rated player wins only about **52.4%** of ladder games, with a CI that includes 50 — so ladder Elo is a **weak** signal here, and band separation must be demonstrated rather than assumed. If bands cannot be distinguished, that is a finding and gets reported as one.

Build order (inputs first, capstone last)

1. **I0 damage engine** — ✅ done.
2. **I1 belief model** + **I4 CHOMP-EV proof** (parallel; both extend `eval_policy` /backtest harnesses).
3. **I2 value/policy net** — imitation → offline RL → self-play, piKL-anchored. The heavy lift.
4. **I3 SLOWKING** preview-Nash on top of I1/I2.
5. **DITTO → PSRO** team-builder.
6. **ALAKAZAM** — assemble I0+I1+I2(+I3) with KL-anchored search. Ship the coach.

Each step: metric + CI + baseline, persisted to JSON, gated in CI. No capstone until its inputs clear their bars.

The per-turn pipeline — WILL, 2026-08-13

This supersedes the dependency graph above as the description of what happens ON A TURN. The graph is still right about what depends on what; it says nothing about the order of operations inside one decision, and that order turns out to carry most of the design. Written from Will's own statement of it, with four corrections argued below. Nothing above is deleted — a prior conclusion is superseded in place and the reason is stated.

What he described

> *"at any point in any open team sheet game, look at its opponent and then ask MAG, hey prune the really bad options. then it should ask DODUO, hey coordinate my moves with both my pokemon and the pokemon i have in the back. then hand those moves to MILTANK and have it run a weighted sample of those possible moves into the rollout engine as many times as time allows for. then have it see the scenarios where it wins (and in poker theory, least bad options) and have it choose from those options (check, raise, fold, all a certain percentage of time) and then play that turn."*

The pipeline as it now stands

stage	model	input	output	fitted?
1. futility gate	**MAG**	the live board	the actions that are not DEAD	**no** — boolean, derived
2. coordination	**DODUO**	surviving actions	scored JOINT actions; the scores ARE the sampling weights	yes
3. playout	**MILTANK**	weighted joint actions	outcomes, played to termination on MEDICHAM	no
4. the mix	**SLOWKING**	the outcome matrix	a frequency per action	no (a solve)
5. the dial	**HYPNO**	the opponent's behaviour	how far to deviate from equilibrium	deferred by Will

Correction 1 — MAG stops scoring and starts gating

Will, 2026-08-13: *"im thinking of cutting mag back in size to say, no fake out after turn 1, no poison on steel, no status moves into gholdengo, no prio moves when farig is on the board... just very simple rules and not the whole convoluted, protect and let my partner ko the threat."*

This resolves a real ordering fault rather than merely simplifying. A learned MAG prunes what is bad ON AVERAGE, and it runs BEFORE DODUO — so it deletes exactly the moves that are bad alone and good in combination, which is the entire class DODUO exists to find. Fake Out into a spread, Follow Me covering a setup turn, a weak click that baits a Protect.

The line that makes early pruning safe:

- **Futile = no partner action rescues it.** Toxic into a Steel is dead whatever the ally clicks. Cut it before coordination; no synergy is lost because none was possible.
- **Bad = conditional.** "Wasteful unless my partner needs the turn" must survive to DODUO.

AND THAT LINE IS NOT SAFE AS FIRST STATED — WILL BROKE IT THE SAME NIGHT. *"of course there is forbidden tech of clicking a prio move when farig is on the board because youre making a prediction of it switching out."* The formulation above quantifies over MY PARTNER'S actions only. It has to quantify over the OPPONENT'S too, and once it does, most of the gate's members stop being unconditional: a priority move into an Armor Tail Farigiraf is dead **only while that Farigiraf is still in the slot when the move resolves.** If they pivot, it connects. The same is true of Toxic into a Steel and of any status into Gholdengo — a move follows the slot, so switching the slot rescues it.

So the gate has two halves and only one is assumption-free:

half	dead because of	pruning it asserts
unconditional — Fake Out out of position, a Choice-locked move, no PP	the USER's own state	nothing
occupant-dependent — status into Gholdengo, poison into Steel, priority into Armor Tail	the TARGET's current identity	*"they will not switch"*

Three of Will's four original examples are in the second half. **The gate may still cut them — it is usually right, and in a format where the read is rare the expected cost is small — but it is a DECLARED PREDICTION and not a rule**, and the plays it deletes are exactly the ones he named.

DEFERRED BY HIS DECISION, NOT MISSED: *"id rather not engage with those type of scenarios at the moment but make a note of it for things to possibly adjust much much later into the project."* Recorded here so the assumption is visible now rather than discovered later as "the bot never makes a read". When it is taken up, the natural home is not the gate at all: an occupant-dependent action is dead under one opponent action and live under another, which is a COLUMN of the stage-4 matrix, not a row to delete before stage 2.

Correction 2 — the gate is DERIVED, never a written list

Every one of the four example rules is wrong as stated, and two of the four are mistakes this project has already made and written down:

the rule as typed	what the format says
no Fake Out after turn 1	`if (source.activeMoveActions > 1) return false` — first turn **out**, not turn 1 of the game. A Fake Out user switching in on turn 6 can Fake Out on turn 6.
no poison on Steel	**Corrosion** — Salazzle, Glimmora.
no status into Gholdengo	Good as Gold is `breakable`, and refuses only when `target !== source`.
no priority vs Farigiraf	Armor Tail is **one of three** abilities (Cud Chew, Armor Tail, Sap Sipper) and exempts Perish Song, Flower Shield, Rototiller.

The category is right every time; the hand-written form is wrong every time. That is the ban-list-of-four shape, and **a prune list is worse than a ban list because it fails silently** — a deleted option leaves no trace in any output.

So the gate does not hold rules. It asks the engine *"does this action produce any effect on this board?"* and drops it if the answer is no. Self-correcting across a regulation change, and it cannot drift from the simulator because it IS the simulator.

OPEN SHEETS DO NOT REMOVE THIS. Will: *"we have open team sheets tho"* — true, and it makes three of the four exact, because the declared ability is known. It does not close the gap:

- **Fake Out's condition is not on any sheet.** `activeMoveActions` is battle STATE. The sheet says the move exists; only the board says whether that body just switched in.
- **The sheet stops being true.** Six legal moves change an ability mid-battle — Skill Swap, Worry Seed, Gastro Acid, Entrainment, Simple Beam, Role Play — plus 11 Mold Breaker carriers and a Mummy. Knock Off has **4,078 corpus uses** and Trick 438, so the item half is routinely false by turn three. This is the existing PREFER OBSERVED OVER DECLARED rule, and it applies to the gate.

The gate therefore reads the **live board**, not the sheet. On turn one those are the same object; by turn four they are not.

THE GATE IS A RANGE, NOT A FILTER — WILL, 2026-08-13, AND THIS IS THE NIGHT'S REAL CORRECTION

He gave two reads in a row and they redefine the stage:

> *"ive seen wolfe do the call out of calling a mon to switch out and using knock off into the new flame orb guts ursaluna" ... "or using a ghost move into a normal type calling a switch" ... "ground into flying etc" ... "thats the world champ difference"*

(The Ursaluna example is Scarlet & Violet and he said so; Ursaluna and Flame Orb are both `isNonstandard: 'Past'` here. Guts and Knock Off are legal, so the SHAPE transfers and that exact board does not. The Ghost-into-Normal and Ground-into-Flying versions are legal and are cleaner anyway, because a type immunity is the most unambiguously dead action there is.)

MEASURED, THIS IS NOT AN EDGE CASE — IT IS THE GATE'S ENTIRE MEMBERSHIP. Derived from the format:

	count	
type immunities	**8**	Dragon→Fairy, Electric→Ground, Fighting→Ghost, Ghost→Normal, Ground→Flying, Normal→Ghost, Poison→Steel, Psychic→Dark

	count	
legal abilities with an immunity-shaped handler	**29**	Levitate*, Volt Absorb, Sap Sipper, Storm Drain, Flash Fire, Good as Gold, Bulletproof, Wonder Guard, Armor Tail, Dazzling, Queenly Majesty, …

(\Levitate reaches this by a hardcoded name in the simulator rather than a handler — ROADMAP #237 — which is its own reason a handler-derived gate would under-count.)

Every one of those is a **switch-read opportunity**, because a move follows the SLOT and a switch replaces the body it lands on. So the gate's most confident cuts are precisely the plays Will is pointing at.

WHAT IS ACTUALLY ASSUMPTION-FREE IS ONLY THE USER'S OWN STATE — six-ish conditions, not a category: Fake Out when not on its first turn out; Choice-locked into a different move; no PP; Disabled, Encored, or a status move under Taunt; Belly Drum at half HP or less. These cannot execute no matter what anybody clicks.

THE FIX IS TO STOP TREATING THE STAGE AS A FILTER.

half	treatment	why
user-state futility	**hard cut**	the action cannot execute; nothing is being predicted
occupant-dependent futility (all 8 + 29)	**weight to near-zero, do not delete**	it is dead *if they stay*; a switch makes it live

In poker terms you do not remove a bluff from your range, you play it 5% of the time. A near-zero weight costs exactly what a deletion costs and keeps the action reachable, so **SLOWKING can discover the right frequency for a Ghost move into a Normal type instead of us hand-deciding it is zero.** That is the argument for having the matrix at all, arriving from a direction nobody planned.

AND IT MOVES THE PREDICTION TO WHERE IT BELONGS. An action that is dead under "they stay" and live under "they switch" is a **cell of the stage-4 matrix**, which is the object built to hold exactly that shape. A row deleted before stage 2 can never become a cell.

Correction 3 — DODUO owns the weights

Will, 2026-08-13: *"yes i think we let doduo do the weights of the remaining options."* Correct, and it is forced: once MAG is boolean it supplies no weights, and the "weighted sample" in stage 3 needs an owner. DODUO scores joint actions; those scores are the weights.

Each stage now has one job and one output type, which is worth more than it looks — a live confusion today is that MAG both prunes and scores, so one number is read for two purposes that want different things.

Two consequences to decide rather than discover. DODUO will now be asked to score joint actions MAG used to hide, and its scores over that wider set are not validated. And it inherits the quarantine: its weights are fitted, so they wait on MEDICHAM. **The futility gate does not** — it has no fitted parameters, which makes it the one piece of this pipeline buildable today.

Correction 4 — the mixture is SOLVED, not sampled

Poker's check/raise/fold frequencies are not chosen from among good outcomes; they are the output of an equilibrium solve. Picking randomly among winning lines is still exploitable, because the mixture was never computed against a best response.

After stage 3 there is a small matrix — order 10x10 joint actions — of (mine x theirs) -> value. Solving it is a tiny linear program. **That solver is SLOWKING and it already exists**: `ismcts.py` is simultaneous-move regret matching that recovers exact Nash, and `solver.py` already does preview Nash plus continual re-solve. The per-turn scope is the unbuilt half, not a new model.

HYPNO is not this stage. It rates the opponent and sets how hard to EXPLOIT — the dial on top of the equilibrium, which is unexploitable and punishes nobody. Deferred by Will's own call, and the only model whose inputs never include MEDICHAM, so the quarantine never blocks it.

AND THE ENDGAME IDEA IS DUSK'S, NOT HYPNO'S — WILL CAUGHT THIS AND HE IS RIGHT. *"wait didnt we have dusknoir as the endgame guy? its hard to keep track."* `docs/MODELS.md:1451` — **DUSK, the endgame tablebase**, EXACT rather than approximate, precomputed offline and shipped as a lookup because it cannot be solved under a turn timer. His description — the opponent reveals their back, find the lines that force a win, hand those down so the rollout steers toward them — is DUSK exactly, because "guaranteed" is what a tablebase gives and what nothing else does.

THE CONFUSION IS IN THESE DOCS, NOT IN HIS MEMORY. THREE THINGS CLAIM THE WORD "ENDGAME". SLOWKING's job line in `docs/MODELS.md` reads *"the endgame — tell you the equilibrium-best move (and win %) on a live position"*, which is indistinguishable in wording from DUSK's entire purpose. That naming collision is why the architecture was hard to hold in one head, and it is fixed here:

model	scope	exact?	when it runs
DUSK	small endgames only	**yes — solved**	offline, then looked up
SLOWKING	any position	no — equilibrium over estimated values	live, per turn
HYPNO	the opponent, not the position	n/a	a deliberate deviation FROM SLOWKING

Why the matrix is kept even if greedy wins

Will, 2026-08-13: *"id still want slowking matrix even if greedy wins right? because then if we play opponents regularly we can adapt and become unexploitable."* **Keeping it is right and is already this project's stated position** — ADR-003 made exploitability the headline because of VGC-Bench: a policy trained on 700,000+ logs that beats a Worlds competitor and is **~100% exploitable**. Winning on average and being safe are different axes.

One correction to the reasoning, because it changes what gets built. Adapting to an opponent makes you MORE exploitable, not less. Nash is the unexploitable object and it never adapts; the moment you deviate to punish a habit you have opened a hole in yourself. What repeated play actually buys is the other direction — a FIXED policy degrades against a repeated opponent, because they learn you, and greedy is a fixed mapping: same board, same click, every time. That is what a regular opponent farms.

So the matrix is not the adaptation mechanism. **It is the floor you stand on and the thing you measure your deviations against**, and HYPNO is the deviation. Deviating from a baseline nobody computed is playing badly with extra steps.

What the measurements say about all of this

The horizon is not a constraint, and I was wrong that it was. Will: *"most games end in like 6 maybe 8 turns."* Measured on the open-sheet store, 13,592 games, forfeits excluded (n=8,593):

p25	median	p75	p90	p99	mean
6	**7**	9	11	16	7.7

67.4% finish by turn 8, **95.3% by turn 12, 99.8% by turn 20** — the rollout horizon. A rollout starts mid-game, so it typically needs four or five more turns.

This is a bigger simplification than it looks. Stage 3 can play to real wins and losses; "the scenarios where it wins" is a signal rather than a proxy. The learned leaf evaluator — **63.70%** against **60.28%** for counting bodies and HP — is load-bearing only on the 0.2% of lines reaching turn

1. It is not a blocker for this design and should not be treated as one. The cost question is

therefore not "how deep" but **how many five-turn playouts fit in a turn of clock**, which is measurable and undone.

A finding nobody was looking for: 4,834 of 13,427 decided games are FORFEITS — 36%, median 5 turns. Over a third of the ladder ends in a concession. That does not touch the pipeline, but anything learning "win probability" from this store is partly modelling *when humans give up*. It must be held separate when MILTANK is evaluated: beating a bot that never concedes is a different measurement.

And the mixing thesis is not yet supported by our own artifact. `data/slowking-eval.json`: greedy minus Nash **0.0409, 95% CI [-0.0001, 0.1735]** — the lower bound does not clear zero. Mixing is not shown to beat greedy at preview scale. It may win per-turn where the action space is far richer, but that is the experiment and not the assumption, exactly as ADR-003 frames it.

The gate that still comes first

None of this starts before MILTANK untimed versus MILTANK on the clock is answered. **31.6% of move decisions were measured as deferred** under time pressure, and "as many times as time allows" is doing load-bearing work in stage 3. If a third of turns do not get the search, this pipeline describes what happens on two turns in three.

And the standing constraint is Will's own, from the same night: *"miltanks rollout needs to just play the game out on medicham and have it match showdown perfectly thats the whole point. miltanks just chooses the actions."* Stage 3 is not permitted to approximate. **The seed is the only place a correct simulator can still produce a wrong game**, which is why ROADMAP #244 is a blocker for this design and not a detail.

Build order that follows

1. **The futility gate.** No fitted parameters, so no quarantine. Buildable now, with a counter for what it pruned and a check that a pruned action really was a no-op — an over-pruning gate deletes a good play and nothing catches it, because both sides of every comparison share the blind spot.
2. **The seeding audit** (#244, #245) — stage 3 cannot be trusted before it.
3. **The clock gate** — decides how wide stages 2-4 are allowed to be.
4. **Per-turn SLOWKING** — the solver exists; the scope is new.
5. **DODUO re-fit** over the wider action set, after MEDICHAM.
6. **HYPNO**, last, and only once there is an equilibrium to deviate FROM.